Lab Manual

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by: Md. Tariquzzaman
Academic Year: 2024–2025

Contents

I Simulating Database Using Python **6**

1 Objective **7**

2 Dataset Description **7**

3 Full Python Solution **7**
3.1 Overview 7

4 Sample Run **9**
4.1 Example Output 9

II Lab Workflow **10**

III DDL and DML **14**

1 Creating a Table **15**
1.1 Common PostgreSQL Data Types 15
1.2 Example: The Product Table 15

2 Dropping Tables **15**

3 Altering Tables **16**

4 Inserting Records **16**

5 Updating Data **16**

6 Deleting Data **16**

7 Retrieval (SELECT) and Filtering **17**
7.1 Logical Operators (AND, OR) 17

IV Integrity Constraints, Sorting, Subqueries, and Data Filtering **18**

1 Sample Database Schema **19**

2 Integrity Constraints **19**
2.1 Primary Key 19
2.2 Foreign Key 20

3 Data Sorting (ORDER BY) 20
3.1 Single Column Sorting 20
3.2 Multiple Column Sorting 20

4 Intermediate SQL: Subqueries 20
4.1 Single-Row Subqueries 20
4.2 Multiple-Row Subqueries 21
4.3 Nested Subqueries (Multiple Nesting) 21
4.4 Correlated Subqueries 22

5 Range Operators 22
5.1 BETWEEN 22
5.2 DISTINCT 22

6 LIMIT 22

V Joins, Aggregation, and Grouping 23

1 Sample Database Schema 24

2 SQL Joins 26
2.1 INNER JOIN 26
2.2 LEFT JOIN (Left Outer Join) 26
2.3 RIGHT JOIN (Right Outer Join) 26
2.4 FULL OUTER JOIN 27
2.5 CROSS JOIN 27
2.6 NATURAL JOIN 28

3 Aggregate Functions 28
3.1 COUNT, SUM, AVG, MIN, MAX 28

4 Grouping Data (GROUP BY) 29
4.1 GROUP BY with Aggregate 29
4.2 GROUP BY with JOIN 29

5 Filtering Groups (HAVING) 29
5.1 HAVING Clause 29

VI Views, Roles, String, Sets and Dates 30

1 Introduction 31

2 Lab Environment Setup 31
2.1 DDL and Data Insertion 31

3 Scalar Functions: String Processing **32**
3.1 Case Conversion . . . 32
3.2 String Sanitization (TRIM) . . . 32
3.3 Substring Extraction and Pattern Matching . . . 33
3.4 Pattern Replacement . . . 33

4 Temporal Data Management **34**
4.1 Intervals and Aging . . . 34
4.2 Field Extraction (EXTRACT) . . . 34

5 Relational Set Operators **35**
5.1 UNION vs UNION ALL . . . 35
5.2 INTERSECT . . . 35
5.3 EXCEPT (Difference) . . . 35

6 Database Views **36**
6.1 Standard Views . . . 36
6.2 Updatable Views and WITH CHECK OPTION . . . 36
6.3 Materialized Views . . . 36

7 Data Control Language (DCL) **38**
7.1 Creating Views . . . 38
7.2 Implementing RBAC . . . 38
7.3 Column-Level Security (Least Privilege) . . . 39
7.4 Revoking Access . . . 39

VII Introduction to Database Design (ERD) **40**

1 Introduction **41**
1.1 Design Phases . . . 41
1.2 Common Design Pitfalls . . . 41

2 The Entity-Relationship (ER) Model **42**
2.1 Entity Sets . . . 42
2.2 Attributes . . . 42

3 Relationships **44**
3.1 Visual Representation . . . 44
3.2 Roles and Recursive Relationships . . . 44
3.3 Attributes on Relationships . . . 45

4 Constraints and Cardinality **46**
4.1 Visual Notation (Arrow Styles) . . . 46
4.2 Types of Mapping Cardinalities . . . 46
4.3 Participation Constraints . . . 47

5 Advanced ER Features **48**
5.1 Weak Entity Sets . . . 48

6 Implementation: Solved Example (Complex ERD to Tables) 49
6.1 The Scenario: University Academic System 49
6.2 Step-by-Step Conversion 49

VIII JDBC 51

1 Lab Workflow 52

2 Database Schema & Data File 54

3 Environment Setup 55
3.1 System Installation 55
3.2 VS Code Configuration 55

4 Understanding JDBC 56
4.1 What is JDBC? 56
4.2 How It Works: The Architecture 56
4.3 Core Components & Syntax 56
4.4 Prepared Statements 56
4.5 Real-Life Application 57

5 JDBC Code Examples 58
5.1 Basic Select (Using Statement) 58
5.2 How to Run the Code 58

IX PL/pgSLQ: Anonymous Blocks 59

1 Lab Workflow 60

2 Database Schema & Data 62

3 Introduction to PL/pgSQL 63
3.1 Anonymous Blocks 63
3.2 Outputting Text 63

4 Variables and Arithmetic 63

5 Single Row Queries (SELECT INTO) 63

6 Control Structures 65
6.1 Conditional Logic (IF/ELSE) 65
6.2 CASE Statement 65

7 Loops 65
7.1 WHILE Loop 65
7.2 FOR Loop (Iterating Query Results) 66

8 Using WITH (CTEs) in Blocks 67

X PL/pgSQL Named Blocks: Functions and Procedures 68

1 Introduction to Named Blocks 69

2 User-Defined Functions (UDFs) 69
2.1 Scalar and Table-Valued Functions 69

3 Stored Procedures 69

4 Parameter Modes and Default Behavior 71
4.1 Implicit Default Behavior 71
4.2 Mode Definitions 71

5 Integration of Parameter Modes 71
5.1 Function Implementation with Mixed Modes 71
5.2 Procedure Implementation with Mixed Modes 72

Lab I

Simulating Database Using Python

1 Objective

The objective of this lab is to understand how basic database operations such as Create, Read, Update, and Delete (CRUD) are implemented internally. This will be achieved by simulating database functionality using a simple text file and Python.

2 Dataset Description

A text file named `students.txt` will be used as the data store. Each record in the file represents one student, with fields separated by a pipe (|) character.

```
-- Format: ID|Name|Age|Major|GPA
1|Alice Johnson|20|Computer Science|3.8
2|Bob Smith|22|Mathematics|3.5
3|Carol Davis|21|Physics|3.9
4|David Brown|23|Computer Science|3.6
```

Listing 1: Sample Database: students.txt

3 Full Python Solution

3.1 Overview

The following Python program implements all CRUD operations and a simple text-based menu interface.

```
# ------------------------------------------------------------
# Basic CRUD System using Text File (students.txt)
# ------------------------------------------------------------

FILENAME = 'students.txt'

# ---------- CREATE ----------
def insert_student(student_id, name, age, major, gpa):
"""Insert a new student record into the file."""
with open(FILENAME, 'a') as file:
file.write(f"{student_id}|{name}|{age}|{major}|{gpa}\n")
print("Record inserted successfully.\n")


# ---------- READ ----------
def read_students():
"""Read and display all records from the file."""
try:
with open(FILENAME, 'r') as file:
print("Student Records:")
for line in file:
if not line.startswith('--'):
print(line.strip())
print()
except FileNotFoundError:
print("Database file not found.\n")
```

```
# ---------- UPDATE ----------
def update_student(student_id, field, new_value):
"""Update a specific student's field."""
lines = []
found = False
with open(FILENAME, 'r') as file:
for line in file:
if line.startswith('--'):
lines.append(line)
continue
data = line.strip().split('|')
if data[0] == str(student_id):
found = True
mapping = {'name':1, 'age':2, 'major':3, 'gpa':4}
if field in mapping:
data[mapping[field]] = str(new_value)
print(f"Updated {field} for ID {student_id}.")
line = '|'.join(data) + '\n'
lines.append(line)
if not found:
print("Record not found.")
else:
with open(FILENAME, 'w') as file:
file.writelines(lines)
print("Record updated successfully.\n")


# ---------- DELETE ----------
def delete_student(student_id):
"""Delete a student record by ID."""
with open(FILENAME, 'r') as file:
lines = file.readlines()

new_lines = [l for l in lines if not (l.split('|')[0] == str(student_id))]

if len(new_lines) == len(lines):
print("Record not found.")
else:
with open(FILENAME, 'w') as file:
file.writelines(new_lines)
print("Record deleted successfully.\n")


# ---------- MENU ----------
def menu():
while True:
print("========== Student Database Menu ==========")
print("1. Insert Student")
print("2. View All Students")
print("3. Update Student")
print("4. Delete Student")
print("5. Exit")
choice = input("Enter your choice: ")

if choice == '1':
student_id = input("Enter ID: ")
name = input("Enter Name: ")
age = input("Enter Age: ")
```

```
major = input("Enter Major: ")
gpa = input("Enter GPA: ")
insert_student(student_id, name, age, major, gpa)
elif choice == '2':
read_students()
elif choice == '3':
student_id = input("Enter ID to update: ")
field = input("Field to update (name/age/major/gpa): ").lower()
new_value = input("Enter new value: ")
update_student(student_id, field, new_value)
elif choice == '4':
student_id = input("Enter ID to delete: ")
delete_student(student_id)
elif choice == '5':
print("Exiting program.")
break
else:
print("Invalid choice.\n")

# Run the menu when script is executed
if __name__ == "__main__":
menu()
```

Listing 2: Full CRUD Implementation in Python

4 Sample Run

4.1 Example Output

```
========== Student Database Menu ==========
1. Insert Student
2. View All Students
3. Update Student
4. Delete Student
5. Exit
Enter your choice: 1
Enter ID: 5
Enter Name: Eve Wilson
Enter Age: 20
Enter Major: Data Science
Enter GPA: 3.9
Record inserted successfully.

Enter your choice: 2
Student Records:
1|Alice Johnson|20|Computer Science|3.8
2|Bob Smith|22|Mathematics|3.5
3|Carol Davis|21|Physics|3.9
4|David Brown|23|Computer Science|3.6
5|Eve Wilson|20|Data Science|3.9
```

Listing 3: Program Output Example

Lab II

Lab Workflow

Lab Workflow

Repeat this subsection for every single lab. If anything in this section fails in IUT lab PCs, check the commands given at the end of the lab manual after notes.

1. Disconnect from the **dsl** connection via nmtui, connect to the ethernet/wired connection via nmtui. If no such connection exists, create an **ethernet** connection and then activate.

2. Download the files from the local server.

3. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. Login to PostgreSQL as ‘postgres‘:

   Log in again to PostgreSQL using ‘postgres‘:

```
sudo -i -u postgres psql
```

   2. Create Your Role and Database

   Important: If the role or database already exists, we must drop them first to avoid conflicts.

```sql
-- Terminate active connections to the database
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'cse4308_12345';

-- Drop database if it exists
DROP DATABASE IF EXISTS cse4308_12345;

-- Drop role if it exists
DROP ROLE IF EXISTS alice;

CREATE ROLE alice LOGIN;
ALTER ROLE alice WITH PASSWORD 'alice123';

CREATE DATABASE cse4308_12345 OWNER alice;

GRANT ALL PRIVILEGES ON DATABASE cse4308_12345 TO alice;
```

   This script will:

   - Terminate active sessions to allow safe dropping.
   - Drop the existing `cse4308_12345` database and `alice` role if present.
   - Create a new `alice` role with a password.
   - Create the database `cse4308_12345` owned by `alice`.
   - Grant all permissions to `alice`.

   3. Exit PostgreSQL:

```
\q
```

4. **Connect Using Your Newly Created User**

   Now connect to your own database as the ‘alice‘ user.

   1. Connect to Your Database:

   Use the following command to log in as the ‘alice‘ user:

```
psql -U alice -d cse4308_12345
```

   2. Verify the Connection:

   Verify that you are connected to the correct database with the following command:

```
\conninfo
```

   Expected output:

```
You are connected to database "cse4308_12345" as user "alice".
```

   3. Exit PostgreSQL:

```
\q
```

5. **Load the Lab SQL File (MUST be run outside psql)**

   Load the SQL file that sets up the tables and inserts initial data.

   1. Load the SQL File:

   Run the following command to load the lab SQL file:

```
psql -U alice -d cse4308_12345 -f /home/cse/Downloads/lab3.sql
```

   This will: - Drop old tables (if they exist). - Create new tables. - Insert initial rows of data.

6. **Reconnect and Verify Tables**

   After loading the SQL file, reconnect to PostgreSQL to verify that the tables have been created.

   1. Reconnect to Your Database:

```
psql -U alice -d cse4308_12345
```

   2. Check Tables:

   Verify the tables have been created by listing them:

```
\dt
```

3. View Table Data:

   Verify the data by running the following SQL queries:

```
SELECT * FROM <table_name>
```

7. Open a text editor and save it in the appropriate submission name format `fullid_CSE4308_L3_T1.sql`. You will use this text editor to write your solutions, make sure the save it after writing each solution. Here is a sample solution file:

```
-- Task 1
INSERT INTO students (student_id, name, major, gpa)
VALUES (104, 'David', 'ME', 3.60);
--Task 2
SELECT name, gpa FROM students WHERE major = 'CSE';
```

Lab III

DDL and DML

Data Definition (DDL)

1 Creating a Table

Creating a table defines the structure of your data. We will create a `PRODUCT` table to track items, their prices, and their categories.

General Syntax:

```
CREATE TABLE table_name
(
attribute1 datatype [ NULL | NOT NULL | UNIQUE ],
attribute2 datatype [ NULL | NOT NULL | UNIQUE ],
...
);
```

1.1 Common PostgreSQL Data Types

- `INT`: Standard whole numbers.
- `VARCHAR(n)`: Variable-length string.
- `SERIAL`: Auto-incrementing integer.
- `NUMERIC(p,s)`: Exact decimals (e.g., Currency).

1.2 Example: The Product Table

We define the table with an ID, Name, Price, and Category.

```
CREATE TABLE PRODUCT
(
PID SERIAL PRIMARY KEY,       -- Auto-incrementing ID
PNAME VARCHAR(50) NOT NULL,   -- Product Name
PRICE INT CHECK (PRICE >= 0),-- Price cannot be negative
CATEGORY VARCHAR(30)          -- Category (e.g., 'Electronics')
);
```

2 Dropping Tables

If you made a mistake and need to delete the table entirely:

```
DROP TABLE PRODUCT;
```

The Cascade Option: If other tables reference the Product table, use `CASCADE`:

```
DROP TABLE PRODUCT CASCADE;
```

3 Altering Tables

We decide to track inventory levels, so we modify the existing table.

```
-- 1. Add a new column for 'STOCK'
ALTER TABLE PRODUCT ADD COLUMN STOCK INT;

-- 2. Remove a column (Example)
ALTER TABLE PRODUCT DROP COLUMN STOCK;

-- 3. Change data type (e.g., make PRICE support decimals)
ALTER TABLE PRODUCT ALTER COLUMN PRICE TYPE NUMERIC(10,2);

-- 4. Rename column
ALTER TABLE PRODUCT RENAME COLUMN PNAME TO PRODUCT_NAME;
```

Data Manipulation (DML)

4 Inserting Records

We will insert data into our ‘PRODUCT‘ table. Note that we are now including the `STOCK` column we added in the previous section.

Named Notation (Recommended):

```
-- Insert a Mouse (Electronics)
INSERT INTO PRODUCT(PID, PNAME, PRICE, CATEGORY, STOCK)
VALUES (DEFAULT, 'Gaming Mouse', 45, 'Electronics', 100);

-- Insert a Pen (Stationery)
INSERT INTO PRODUCT(PID, PNAME, PRICE, CATEGORY, STOCK)
VALUES (DEFAULT, 'Gel Pen', 5, 'Stationery', 500);
```

5 Updating Data

We want to update the price of the ’Gaming Mouse’. **Warning:** Always use a `WHERE` clause.

```
UPDATE PRODUCT
SET PRICE = 40      -- Discounting the price
WHERE PID = 1;      -- Only for the item with ID 1
```

6 Deleting Data

We stop selling items that have 0 stock.

```
DELETE FROM PRODUCT
WHERE STOCK = 0;
```

7 Retrieval (SELECT) and Filtering

The core of SQL is querying data using the columns we created.

Basic Filtering:

```
-- Find expensive products
SELECT * FROM PRODUCT WHERE PRICE > 50;
```

7.1 Logical Operators (AND, OR)

To answer complex questions, we combine conditions.

1. **AND Operator:** Both conditions must be true.

```
-- Find products that are expensive AND in 'Electronics'
SELECT * FROM PRODUCT
WHERE PRICE > 30 AND CATEGORY = 'Electronics';
```

2. **OR Operator:** At least one condition must be true.

```
-- Find products that are either 'Electronics' OR 'Stationery'
SELECT * FROM PRODUCT
WHERE CATEGORY = 'Electronics' OR CATEGORY = 'Stationery';
```

Lab IV

Integrity Constraints, Sorting, Subqueries, and Data Filtering

1 Sample Database Schema

To demonstrate the concepts in this lab, we will refer to the following three tables: **Department**, **Employee**, and **Project**.

Table 1: Department

dept_id (PK)	**dept_name**	**location**
D01	CSE	Building A
D02	EEE	Building B
D03	BBA	Building C

Table 2: Employee

emp_id (PK)	**name**	**salary**	**dept_id** (FK)
E101	Alice	50000	D01
E102	Bob	60000	D01
E103	Charlie	55000	D02
E104	David	45000	D03
E105	Eve	70000	D01

Table 3: Project

proj_id (PK)	**proj_name**	**dept_id** (FK)
P01	AI Research	D01
P02	Circuit Design	D02
P03	Web Dev	D01

2 Integrity Constraints

2.1 Primary Key

A Primary Key is a column (or a set of columns) that uniquely identifies each row in a table. It must contain unique values and cannot contain NULL values.

```sql
CREATE TABLE Department (
  dept_id VARCHAR(5),
  dept_name VARCHAR(20),
  location VARCHAR(50),
  CONSTRAINT pk_department PRIMARY KEY (dept_id)
);
```

2.2 Foreign Key

Foreign keys are used to enforce referential integrity between two tables. A foreign key in one table references the primary key of another table.

```
CREATE TABLE Employee (
  emp_id VARCHAR(5),
  name VARCHAR(30),
  salary NUMERIC(10, 2),
  dept_id VARCHAR(5),
  CONSTRAINT pk_employee PRIMARY KEY (emp_id),
  CONSTRAINT fk_emp_dept FOREIGN KEY (dept_id)
  REFERENCES Department(dept_id)
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
```

Note: The `ON DELETE CASCADE` clause ensures that if a Department is deleted, all Employees belonging to that department are also automatically deleted.

3 Data Sorting (ORDER BY)

The `ORDER BY` clause is used to sort the result set in either ascending (ASC) or descending (DESC) order.

3.1 Single Column Sorting

Retrieve employees sorted by salary in descending order.

```
SELECT name, salary
FROM Employee
ORDER BY salary DESC;
```

3.2 Multiple Column Sorting

Sort by department ID first, then by salary within that department.

```
SELECT dept_id, name, salary
FROM Employee
ORDER BY dept_id ASC, salary DESC;
```

4 Intermediate SQL: Subqueries

A subquery (also known as a nested query or inner query) is a query within another SQL query.

4.1 Single-Row Subqueries

These subqueries return a single value (one row, one column). They are typically used with comparison operators ($=, >, <, >=, <=, <>$).

Example: Find employees who earn more than the average salary of all employees.

```
SELECT name, salary
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);
```

4.2 Multiple-Row Subqueries

These subqueries return more than one row. They are used with set operators like `IN`, `ANY`, or `ALL`.

Example (IN): Find employees who work in departments located in 'Building A'.

```
SELECT name
FROM Employee
WHERE dept_id IN (
  SELECT dept_id
  FROM Department
  WHERE location = 'Building A'
);
```

4.3 Nested Subqueries (Multiple Nesting)

SQL allows nesting subqueries inside other subqueries.

Example: Find the name of employees who work in a department that has an active project named 'AI Research'.

Logic: Find the project's department → Find that department in the Employee table.

```
SELECT name
FROM Employee
WHERE dept_id IN (
  SELECT dept_id
  FROM Project
  WHERE proj_name = 'AI Research'
);
```

You can also use "NOT IN" to find the people who are not in those results

Example (Deep Nesting): Find employees who work in a department where the department ID exists in the Project table, but only for projects managed by the 'CSE' department.

```
SELECT name
FROM Employee
WHERE dept_id IN (
  SELECT dept_id
  FROM Project
  WHERE dept_id = (
    SELECT dept_id
    FROM Department
    WHERE dept_name = 'CSE'
  )
);
```

4.4 Correlated Subqueries

A correlated subquery depends on the outer query for its values. It is executed once for *each row* selected by the outer query.

Example: Find employees whose salary is higher than the average salary **of their specific department**.

```
SELECT e1.name, e1.salary, e1.dept_id
FROM Employee e1
WHERE e1.salary > (
  SELECT AVG(e2.salary)
  FROM Employee e2
  WHERE e2.dept_id = e1.dept_id
);
```

5 Range Operators

5.1 BETWEEN

```
SELECT name, salary
FROM Employee
WHERE salary BETWEEN 45000 AND 60000;
```

5.2 DISTINCT

```
-- Returns unique department IDs from the Employee table
SELECT DISTINCT dept_id FROM Employee;
```

6 LIMIT

Used for pagination or restricting results.

```
-- Top 3 highest paid employees
SELECT name, salary
FROM Employee
ORDER BY salary DESC
LIMIT 3;
```

Lab V

Joins, Aggregation, and Grouping

1 Sample Database Schema

To demonstrate the concepts in this lab, we will refer to the following three tables: **Department**, **Employee**, and **Project**.

Table 1: Department

dept_id (PK)	**dept_name**	**location**
D01	CSE	Building A
D02	EEE	Building B
D03	BBA	Building C

Table 2: Employee

emp_id (PK)	**name**	**salary**	**dept_id** (FK)
E101	Alice	50000	D01
E102	Bob	60000	D01
E103	Charlie	55000	D02
E104	David	45000	D03
E105	Eve	70000	D01

Table 3: Project

proj_id (PK)	**proj_name**	**dept_id** (FK)
P01	AI Research	D01
P02	Circuit Design	D02
P03	Web Dev	D01

Database Setup Code

Run the following SQL block to create the tables and insert the sample data used in the examples below.

```sql
-- 1. Clean up existing tables
DROP TABLE IF EXISTS Project CASCADE;
DROP TABLE IF EXISTS Employee CASCADE;
DROP TABLE IF EXISTS Department CASCADE;

-- 2. Create the Department Table
CREATE TABLE Department (
dept_id VARCHAR(5),
dept_name VARCHAR(20),
location VARCHAR(50),
CONSTRAINT pk_department PRIMARY KEY (dept_id)
);
```

```sql
-- 3. Create the Employee Table
CREATE TABLE Employee (
emp_id VARCHAR(5),
name VARCHAR(30),
salary NUMERIC(10, 2),
dept_id VARCHAR(5),
CONSTRAINT pk_employee PRIMARY KEY (emp_id),
CONSTRAINT fk_emp_dept FOREIGN KEY (dept_id)
REFERENCES Department(dept_id)
ON DELETE CASCADE
ON UPDATE CASCADE
);

-- 4. Create the Project Table
CREATE TABLE Project (
proj_id VARCHAR(5),
proj_name VARCHAR(50),
dept_id VARCHAR(5),
CONSTRAINT pk_project PRIMARY KEY (proj_id),
CONSTRAINT fk_proj_dept FOREIGN KEY (dept_id)
REFERENCES Department(dept_id)
ON DELETE CASCADE
ON UPDATE CASCADE
);

-- 5. Insert Data into Department
INSERT INTO Department (dept_id, dept_name, location) VALUES
('D01', 'CSE', 'Building A'),
('D02', 'EEE', 'Building B'),
('D03', 'BBA', 'Building C');

-- 6. Insert Data into Employee
INSERT INTO Employee (emp_id, name, salary, dept_id) VALUES
('E101', 'Alice', 50000, 'D01'),
('E102', 'Bob', 60000, 'D01'),
('E103', 'Charlie', 55000, 'D02'),
('E104', 'David', 45000, 'D03'),
('E105', 'Eve', 70000, 'D01');

-- 7. Insert Data into Project
INSERT INTO Project (proj_id, proj_name, dept_id) VALUES
('P01', 'AI Research', 'D01'),
('P02', 'Circuit Design', 'D02'),
('P03', 'Web Dev', 'D01');
```

2 SQL Joins

Joins allow us to combine rows from two or more tables based on a related column.

2.1 INNER JOIN

The `INNER JOIN` returns records that have matching values in **both** tables.

Query: Get Employee names and their Department location.

```
SELECT E.name, D.dept_name, D.location
FROM Employee E
INNER JOIN Department D ON E.dept_id = D.dept_id;
```

Result:

name	dept_name	location
Alice	CSE	Building A
Bob	CSE	Building A
Charlie	EEE	Building B
David	BBA	Building C
Eve	CSE	Building A

2.2 LEFT JOIN (Left Outer Join)

Returns all records from the left table, and the matched records from the right table. If there is no match, the right side will contain NULL.

Query: Show all Departments and their associated Projects (if any).

```
SELECT D.dept_id, D.dept_name, P.proj_name
FROM Department D
LEFT JOIN Project P ON D.dept_id = P.dept_id;
```

Result: *Note that D03 (BBA) has no project, so returns NULL.*

dept_id	dept_name	proj_name
D01	CSE	AI Research
D01	CSE	Web Dev
D02	EEE	Circuit Design
D03	BBA	*NULL*

2.3 RIGHT JOIN (Right Outer Join)

Returns all records from the right table, and the matched records from the left table.

Query: Show all Projects and the Department they belong to.

```
SELECT P.proj_name, D.dept_name
FROM Project P
RIGHT JOIN Department D ON P.dept_id = D.dept_id;
```

Result:

proj_name	dept_name
AI Research	CSE
Web Dev	CSE
Circuit Design	EEE
NULL	BBA

2.4 FULL OUTER JOIN

Returns all records when there is a match in either left or right table.

Query: Match Employees to Projects based on Department ID.

```
SELECT E.name, P.proj_name
FROM Employee E
FULL OUTER JOIN Project P ON E.dept_id = P.dept_id;
```

Result: *David (D03) has no project match.*

name	proj_name
Alice	AI Research
Alice	Web Dev
Bob	AI Research
Bob	Web Dev
Eve	AI Research
Eve	Web Dev
Charlie	Circuit Design
David	*NULL*

2.5 CROSS JOIN

Produces a Cartesian product (every row of Table 1 combined with every row of Table 2).

Query: Combine every Department with every Project (3 Depts $\times$ 3 Projects $=$ 9 Rows).

```
SELECT D.dept_name, P.proj_name
FROM Department D
CROSS JOIN Project P;
```

Result:

dept_name	proj_name
CSE	AI Research
CSE	Circuit Design
CSE	Web Dev
EEE	AI Research
EEE	Circuit Design
EEE	Web Dev
BBA	AI Research
BBA	Circuit Design
BBA	Web Dev

2.6 NATURAL JOIN

A `NATURAL JOIN` creates an implicit join based on all columns in the two tables that have the same name and data type. In our schema, both tables share the `dept_id` column.

Query: Join Employee and Department automatically on the shared `dept_id`.

```sql
SELECT name, dept_name, location
FROM Employee
NATURAL JOIN Department;
```

Result: *Note: explicit ON clauses are not required.*

name	dept_name	location
Alice	CSE	Building A
Bob	CSE	Building A
Charlie	EEE	Building B
David	BBA	Building C
Eve	CSE	Building A

3 Aggregate Functions

3.1 COUNT, SUM, AVG, MIN, MAX

Query: Calculate statistics on the Employee table.

```sql
SELECT
COUNT(*) AS total_staff,
SUM(salary) AS total_cost,
AVG(salary) AS avg_pay,
MIN(salary) AS min_pay,
MAX(salary) AS max_pay
FROM Employee;
```

Result:

total_staff	total_cost	avg_pay	min_pay	max_pay
5	280000	56000	45000	70000

4 Grouping Data (GROUP BY)

4.1 GROUP BY with Aggregate

Query: Count the number of employees in each department.

```
SELECT dept_id, COUNT(*) AS emp_count
FROM Employee
GROUP BY dept_id;
```

Result:

dept_id	emp_count
D01	3
D02	1
D03	1

4.2 GROUP BY with JOIN

Query: Find the total salary expense per Department Name.

```
SELECT D.dept_name, SUM(E.salary) AS total_salary
FROM Department D
JOIN Employee E ON D.dept_id = E.dept_id
GROUP BY D.dept_name;
```

Result:

dept_name	total_salary
CSE	180000
EEE	55000
BBA	45000

5 Filtering Groups (HAVING)

The `HAVING` clause filters groups **after** the aggregation has occurred.

5.1 HAVING Clause

Query: Find departments where the average salary is greater than 50,000.

```
SELECT dept_id, AVG(salary) AS avg_sal
FROM Employee
GROUP BY dept_id
HAVING AVG(salary) > 50000;
```

Result: *D03 (BBA) is excluded because its average (45,000) is not > 50,000.*

dept_id	avg_sal
D01	60000.00
D02	55000.00

Lab VI

Views, Roles, String, Sets and Dates

1 Introduction

This lab module demonstrates advanced data manipulation techniques in PostgreSQL. We will cover character string processing for data cleaning, temporal operations for date logic, set-theoretic operators for combining datasets, and Data Control Language (DCL) for security management.

2 Lab Environment Setup

Before executing the functional demonstrations, you must create the schema and populate the tables. Execute the following SQL block to initialize your workspace.

2.1 DDL and Data Insertion

```sql
-- 1. AcademicStaff Table
CREATE TABLE AcademicStaff (
id SERIAL PRIMARY KEY,
full_name VARCHAR(100),
email_addr VARCHAR(100),
joining_date DATE
);

INSERT INTO AcademicStaff (id, full_name, email_addr, joining_date) VALUES
(101, 'ALICE munn', 'alice.m@university.edu', '2018-01-15'),
(102, 'bob roberts', 'bob.roberts@gmail.com', '2019-11-20'),
(103, 'Dr. chaRLie', 'charlie.c@yahoo.com', '2020-03-10'),
(104, 'DAVID smith', 'david.s@university.edu', '2021-07-01'),
(105, 'eve O''Connor', 'eve.oconnor@outlook.com', '2022-02-14'),
(106, 'frank LIN', 'frank.lin@protonmail.com', '2023-08-30'),
(107, 'Grace HOPPER', 'g.hopper@cs.edu', '2024-01-05'),
(108, 'helen keller', 'helen.k@org.net', '2024-09-12');

-- 2. SystemLogs Table
CREATE TABLE SystemLogs (
log_id INT PRIMARY KEY,
raw_entry TEXT
);

INSERT INTO SystemLogs (log_id, raw_entry) VALUES
(1, '   System_Start   '),
(2, '##CRITICAL_ERROR##'),
(3, 'User_Login_Success'),
(4, '   WARN: Low_Memory '),
(5, '__Connection_Timeout__');

-- 3. Course Enrollment Tables
CREATE TABLE Course_Alpha (student_id INT);
CREATE TABLE Course_Beta (student_id INT);

INSERT INTO Course_Alpha (student_id) VALUES
(1001), (1002), (1003), (1004), (1005);

INSERT INTO Course_Beta (student_id) VALUES
(1004), (1005), (1006), (1007), (1008);
```

Listing 4: Environment Setup Script

3 Scalar Functions: String Processing

String functions are essential for *Data Normalization*—the process of organizing data to appear similar across all records.

3.1 Case Conversion

Inconsistent casing often occurs during manual data entry.

- `UPPER(str)` / `LOWER(str)`: Converts entire string case.
- `INITCAP(str)`: Capitalizes the first letter of each word (Title Case).

```sql
SELECT
full_name AS original,
UPPER(full_name) AS upper_case,
INITCAP(full_name) AS normalized_case
FROM AcademicStaff
WHERE id <= 104;
```

Result Set:

original	upper_case	normalized_case
ALICE munn	ALICE MUNN	Alice Munn
bob roberts	BOB ROBERTS	Bob Roberts
Dr. chaRLie	DR. CHARLIE	Dr. Charlie
DAVID smith	DAVID SMITH	David Smith

3.2 String Sanitization (TRIM)

Sanitization removes unwanted characters, such as leading spaces or specific symbols.

- `TRIM([BOTH|LEADING|TRAILING] 'char' FROM str)`: If 'char' is omitted, whitespace is removed.

```sql
SELECT
raw_entry,
TRIM(raw_entry) AS standard_trim,
TRIM(BOTH '#' FROM raw_entry) AS hash_removal
FROM SystemLogs
WHERE log_id IN (1, 2, 4);
```

Result Set:

raw_entry	standard_trim	hash_removal
' System_Start '	'System_Start'	' System_Start '
'##CRITICAL_ERROR##'	'##CRITICAL_ERROR##'	'CRITICAL_ERROR'
' WARN: Low_Memory '	'WARN: Low_Memory'	' WARN: Low_Memory '

3.3 Substring Extraction and Pattern Matching

Used to parse specific information from larger strings (e.g., extracting domains from emails).

- `SUBSTR(str, start, length)`: Extracts segment starting at index *start*.
- `LEFT(str, n)` / `RIGHT(str, n)`: Takes n characters from ends.
- `POSITION(substring IN string)`: Returns the index of a character.

```sql
-- We use POSITION to find the '@' symbol, then extract everything after it.
SELECT
email_addr,
POSITION('@' IN email_addr) as at_symbol_index,
SUBSTR(email_addr, POSITION('@' IN email_addr) + 1) AS domain_only
FROM AcademicStaff
WHERE id BETWEEN 101 AND 103;
```

Listing 5: Dynamic Domain Extraction

Result Set:

email_addr	at_symbol_index	domain_only
alice.m@university.edu	8	university.edu
bob.roberts@gmail.com	12	gmail.com
charlie.c@yahoo.com	10	yahoo.com

3.4 Pattern Replacement

```sql
SELECT
raw_entry,
REPLACE(raw_entry, '_', ' ') AS clean_text
FROM SystemLogs
WHERE log_id = 3;
```

Result: Converts `User_Login_Success` → `User Login Success`.

4 Temporal Data Management

4.1 Intervals and Aging

PostgreSQL provides the `AGE` function to calculate the symbolic difference between two dates.

```
-- Calculating Tenure. Assumed Current Date: 2025-01-01
SELECT
INITCAP(full_name) AS name,
AGE('2025-01-01', joining_date) AS tenure
FROM AcademicStaff
WHERE id >= 106;
```

Result Set:

name	tenure
Frank Lin	1 year 4 mons 2 days
Grace Hopper	11 mons 27 days
Helen Keller	3 mons 20 days

4.2 Field Extraction (EXTRACT)

The `EXTRACT` function retrieves sub-fields (Year, Month, Day, Quarter, Week) from a date source. This is useful for aggregation (e.g., "Sales per Quarter").

```
SELECT
joining_date,
EXTRACT(YEAR FROM joining_date) AS join_year,
EXTRACT(QUARTER FROM joining_date) AS fiscal_qtr
FROM AcademicStaff
WHERE id <= 104;
```

5 Relational Set Operators

Set operators combine the result sets of two or more queries. **Rules for Set Operations:**

1. **Union Compatibility:** The queries must return the same number of columns.
2. **Type Compatibility:** Corresponding columns must have compatible data types.

5.1 UNION vs UNION ALL

- `UNION`: Combines sets and **removes duplicates**. (Logical OR)
- `UNION ALL`: Combines sets and **keeps duplicates**. Faster performance.

```
-- Returns unique list of all students enrolled in ANY course
SELECT student_id FROM Course_Alpha
UNION
SELECT student_id FROM Course_Beta;
```

Result Count: 8 Rows (1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008).

5.2 INTERSECT

Returns only rows that appear in **BOTH** result sets. (Logical AND).

```
-- Find students taking BOTH courses
SELECT student_id FROM Course_Alpha
INTERSECT
SELECT student_id FROM Course_Beta;
```

Result Set:

student_id
1004
1005

5.3 EXCEPT (Difference)

Returns rows from the first query that are **NOT** present in the second query. Note: In Oracle, this is called `MINUS`.

```
-- Students in Alpha who are NOT taking Beta
SELECT student_id FROM Course_Alpha
EXCEPT
SELECT student_id FROM Course_Beta;
```

Result Set:

student_id
1001
1002
1003

6 Database Views

A View is a "virtual table" defined by a query. It does not store data itself (unless Materialized) but fetches it from the base tables when queried. Views are critical for simplifying complex queries and implementing security by hiding sensitive columns.

6.1 Standard Views

```sql
-- Create a view that hides the ID and raw Joining Date
CREATE VIEW View_PublicDirectory AS
SELECT
INITCAP(full_name) AS display_name,
email_addr
FROM AcademicStaff;

-- Query the View just like a table
SELECT * FROM View_PublicDirectory WHERE email_addr LIKE '%edu';
```

6.2 Updatable Views and WITH CHECK OPTION

In PostgreSQL, simple views (1:1 mapping to a table without aggregates) are automatically updatable. The `WITH CHECK OPTION` clause is a powerful constraint that prevents a user from performing an update or insert through the view if the new row would vanish from the view (i.e., it violates the view's `WHERE` clause).

```sql
-- 1. Create a Restricted View for 'edu' emails only
CREATE OR REPLACE VIEW View_Edu_Staff AS
SELECT id, full_name, email_addr
FROM AcademicStaff
WHERE email_addr LIKE '%edu'
WITH CHECK OPTION;

-- 2. Valid Insert (Succeeds)
INSERT INTO View_Edu_Staff (full_name, email_addr)
VALUES ('John Doe', 'john.d@university.edu');

-- 3. Invalid Insert (Fails due to Check Option)
-- Fails because the email does not end in 'edu', so the row
-- would effectively disappear from this view immediately.
INSERT INTO View_Edu_Staff (full_name, email_addr)
VALUES ('Jane Doe', 'jane.d@gmail.com');
```

6.3 Materialized Views

Unlike standard views, a **Materialized View** physically saves the result of the query to the disk. This drastically improves read performance for expensive queries (e.g., complex joins or aggregations) but requires manual refreshing when the base data changes.

```sql
-- Define a heavy analytical query physically
CREATE MATERIALIZED VIEW MatView_StaffAnalysis AS
SELECT
EXTRACT(YEAR FROM joining_date) as year,
COUNT(*) as total_recruits
```

```sql
FROM AcademicStaff
GROUP BY EXTRACT(YEAR FROM joining_date);

-- Querying is instant (no recalculation)
SELECT * FROM MatView_StaffAnalysis;

-- Refreshing the data after base table updates
REFRESH MATERIALIZED VIEW MatView_StaffAnalysis;
```

7 Data Control Language (DCL)

DCL handles permissions and access control. In a production environment, permissions are rarely assigned to individual users; instead, **Role-Based Access Control (RBAC)** is used via Group Roles.

7.1 Creating Views

A view can restrict what columns a user sees, serving as a first layer of security.

```
CREATE OR REPLACE VIEW View_Student_Public AS
SELECT name, cgpa
FROM Students;

-- Verify the view
SELECT * FROM View_Student_Public;
```

7.2 Implementing RBAC

In a real system, you create **Roles** based on job functions (Group Roles), grant permissions to those roles, and then assign users to them. This simplifies management compared to granting permissions to every individual user.

7.2.1 Step 1: Create Functional Roles

We create abstract roles that cannot log in (`NOLOGIN`) to act as containers for permissions.

```
-- 1. Read-Only Role (e.g., for general staff)
CREATE ROLE role_readonly NOLOGIN;
GRANT SELECT ON Depts TO role_readonly;
GRANT SELECT ON View_Student_Public TO role_readonly;

-- 2. Marks Entry Role (e.g., for Teachers)
CREATE ROLE role_marks_entry NOLOGIN;

-- Teachers need to read basic info AND update grades
-- We use Role Inheritance here:
GRANT role_readonly TO role_marks_entry;
GRANT INSERT, UPDATE ON Grades TO role_marks_entry;
```

7.2.2 Step 2: Create Users and Assign Roles

We create concrete users (which imply `LOGIN`) and assign them to the functional roles.

```
-- Create concrete users
CREATE USER user_teacher WITH PASSWORD 'pass123';
CREATE USER user_admin WITH PASSWORD 'pass123';

-- Assign the appropriate role
GRANT role_marks_entry TO user_teacher;

-- user_teacher now inherits:
-- 1. SELECT on Depts (via role_readonly)
-- 2. SELECT on View_Student_Public (via role_readonly)
```

```
-- 3. INSERT/UPDATE on Grades (via role_marks_entry)
```

7.2.3 Step 3: Verification (Impersonation)

You can switch roles in the session to test permissions without logging out.

```
SET ROLE user_teacher;

-- Should Succeed
SELECT * FROM Depts;

-- Should Fail (No permission on raw Student table, only View)
SELECT * FROM Students;

-- Reset to original superuser
RESET ROLE;
```

7.3 Column-Level Security (Least Privilege)

You can restrict access to specific columns. For example, a user might need to verify a log entry exists but should not be allowed to see sensitive content.

```
-- Create a restricted auditor role
CREATE ROLE auditor NOLOGIN;

-- Allow reading ONLY the log_id, effectively hiding raw_entry
GRANT SELECT (log_id) ON SystemLogs TO auditor;
```

7.4 Revoking Access

Privileges can be removed from a role, or a user can be removed from a group.

```
-- Revoke specific privilege from a role
REVOKE INSERT ON Grades FROM role_marks_entry;

-- Remove a user from a group
REVOKE role_marks_entry FROM user_teacher;
```

Lab VII

Introduction to Database Design (ERD)

1 Introduction

The database design process is a structured approach used to transform real-world data requirements into a physical database implementation. It is crucial to ensure the final system is efficient, consistent, and capable of supporting user needs.

1.1 Design Phases

The design process is typically divided into three major phases:

1. **Initial Phase (Requirement Analysis):** This phase involves characterizing the data needs of prospective database users. The designer interacts with domain experts to generate a specification of user requirements. For instance, in a university context, this might involve identifying that the system needs to store data about students, instructors, and the courses they take.

2. **Conceptual-Design Phase:** Here, the designer translates the requirements into a conceptual schema. The **Entity-Relationship (ER) model** is the standard tool used for this purpose. The result is an ER Diagram (ERD), which is an abstract representation of the data, independent of the specific database software (like MySQL or Oracle) that will eventually be used.

3. **Implementation Phase:** This phase moves from the abstract to the concrete:

   - **Logical Design:** Maps the conceptual ER schema onto the implementation data model (usually the Relational Model, consisting of tables and constraints).
   - **Physical Design:** Handles storage details, such as file organization and indexing, to optimize performance.

1.2 Common Design Pitfalls

A robust design must avoid two specific errors:

- **Redundancy:** This occurs when the same piece of information is stored in multiple places. It is dangerous because updating the data in one location without updating it in others leads to *data inconsistency*.

- **Incompleteness:** A design is incomplete if it fails to model necessary aspects of the enterprise or introduces bad business logic, making it impossible to store required information.

2 The Entity-Relationship (ER) Model

The ER model represents data as a collection of entities and relationships.

2.1 Entity Sets

An **entity** is a distinct object in the real world that is distinguishable from all other objects. Examples include a specific person ("Einstein"), a company, or a course.

An **entity set** is a collection of entities of the same type that share the same properties (e.g., the set of all students).

Visual Representation:

- **Rectangles** represent entity sets.
- The name of the entity set is placed at the top or center.
- **Attributes** are listed inside the rectangle.
- The **Primary Key** (the attribute that uniquely identifies an entity) is underlined.

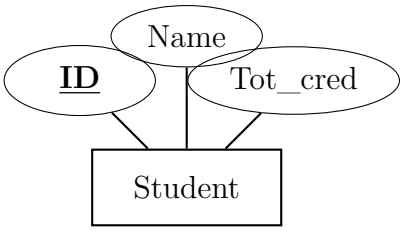

Figure 1: Example of an Entity Set. The attribute ID is the primary key.

2.2 Attributes

Attributes describe the properties of an entity. They are classified as follows:

2.2.1 1. Simple vs. Composite Attributes

- **Simple Attribute:** An attribute that cannot be divided further (e.g., `ID`, `Salary`).
- **Composite Attribute:** An attribute that has meaningful sub-parts. For example, a `Name` attribute might be composed of `First Name`, `Middle Initial`, and `Last Name`.
- **ERD Notation:** Composite attributes are connected to their parent attribute.

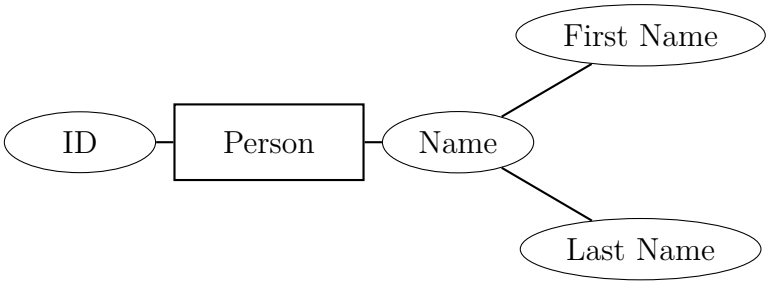

Figure 2: Simple Attribute (ID) vs. Composite Attribute (Name broken into parts).

2.2.2 2. Single-valued vs. Multi-valued Attributes

- **Single-valued:** The attribute holds a single value for a specific entity.
- **Multi-valued:** The attribute may hold multiple values (e.g., phone numbers). Multi-valued attribute allows for multiple entries for the same property.
- **ERD Notation:** Enclosed in a **double ellipse**.

Figure 3: Single-valued (ID) vs. Multi-valued Attribute (Phone), denoted by a double ellipse.

2.2.3 3. Derived Attributes

- The value is computed from other attributes (e.g., `Age` from `Date_of_Birth`).
- **ERD Notation:** Enclosed in a **dashed ellipse**.

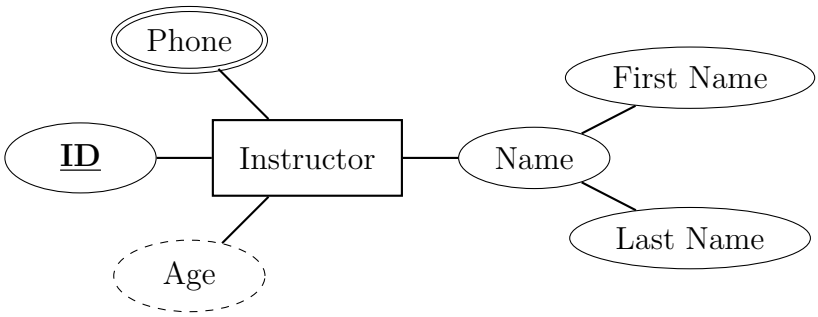

Figure 4: Entity with Composite (Name), Multi-valued (Phone), and Derived (Age) attributes.

3 Relationships

A **relationship** is an association between entities. For example, an `advisor` relationship links a `Student` entity with an `Instructor` entity.

3.1 Visual Representation

- **Diamonds** represent relationship sets.
- Solid lines connect the diamond to the participating entity sets.

Figure 5: A simple binary relationship connecting Instructor and Student.

3.2 Roles and Recursive Relationships

3.2.1 Concept Definition

In most relationships, we connect two *different* entity sets (e.g., a `Student` takes a `Course`). However, sometimes an entity set needs to relate to **itself**. This is called a **recursive relationship**.

Real-world Logic: Imagine a single table containing a list of all University Courses.

- *Course A* (Introduction to C) is a prerequisite for *Course B* (Data Structures).
- Both A and B belong to the same entity set: **Course**.
- We are linking a record in the `Course` entity to another record in the *same* `Course` entity.

3.2.2 The Necessity of Roles

Since both participants in the relationship are the same entity set, the meaning of the relationship can become ambiguous. To solve this, we use **Roles**.

- Roles are labels placed on the relationship lines to specify *how* the entity participates.
- In the prerequisite example, one instance of the course plays the role of the **Main Course** (the one having a requirement), and the other plays the role of the **Prerequisite** (the requirement itself).

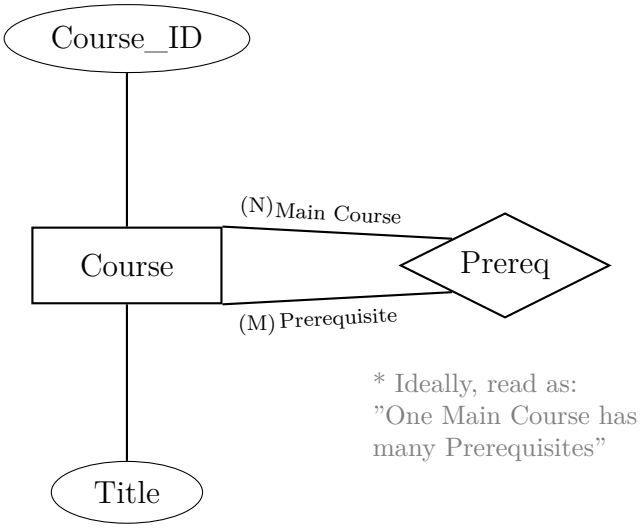

Figure 6: Recursive Relationship: The `Course` entity participates twice. Top line represents the course taking the requirement; bottom line represents the requirement itself.

3.2.3 Implementation Note

When mapping this to a database table (Implementation Phase), a recursive relationship typically results in a **Self-Join**. You would add a Foreign Key column (e.g., `prereq_id`) to the `Course` table that references the `course_id` of the same table.

3.3 Attributes on Relationships

Attributes can also be assigned to relationships. For instance, in an `Advisor` relationship, we might need to track the `date` the student was assigned to that instructor. This attribute is attached to the diamond.

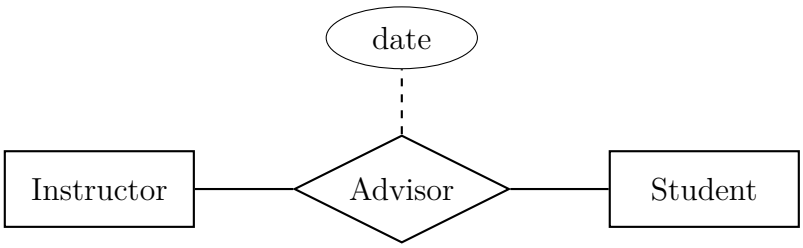

Figure 7: A relationship attribute (date) attached to the Advisor relationship.

4 Constraints and Cardinality

Mapping cardinality defines the number of entities to which another entity can be associated via a relationship set.

4.1 Visual Notation (Arrow Styles)

In standard ER diagrams, the style of the line tip indicates cardinality:

- **Directed Line (→):** Represents "One".
- **Undirected Line (—):** Represents "Many" (zero or more).

4.2 Types of Mapping Cardinalities

Type	Description	Diagram Style
One-to-One	An entity in A is associated with at most one in B, and vice versa.	→ on both sides
One-to-Many	An entity in A is associated with many in B. An entity in B is associated with at most one in A.	→ on A, — on B
Many-to-One	An entity in A is associated with at most one in B. An entity in B is associated with many in A.	— on A, → on B
Many-to-Many	Entities in both sets can be associated with any number of entities in the other set.	— on both sides

Table 1: Cardinality Mapping Styles.

Figure 8: **One-to-One**: A student has at most one advisor, and an instructor advises at most one student.

Figure 9: **One-to-Many**: An instructor can advise many students, but a student has only one advisor.

Figure 10: **Many-to-One**: An instructor is associated with at most one student, but a student may have many instructors.

Figure 11: **Many-to-Many**: An instructor advises many students, and a student has many advisors (e.g., joint supervision).

4.3 Participation Constraints

- **Total Participation:** Every entity in the set *must* participate in the relationship. This is indicated by a **double line**.
- **Partial Participation:** Some entities may not participate. This is indicated by a **single line**.

Figure 12: **Total Participation**: The double line connecting `Student` and `Advisor` indicates that every student must have an associated instructor.

5 Advanced ER Features

5.1 Weak Entity Sets

A **weak entity** is an entity that cannot be uniquely identified by its own attributes alone. It typically represents a dependency.

- It must be associated with a **strong entity** (owner entity).
- It uses a **discriminator** (partial key) combined with the strong entity's primary key for unique identification.

Diagram Notation:

- **Weak Entity Set:** Double Rectangle.
- **Identifying Relationship:** Double Diamond.
- **Discriminator Attribute:** Dashed Underline.

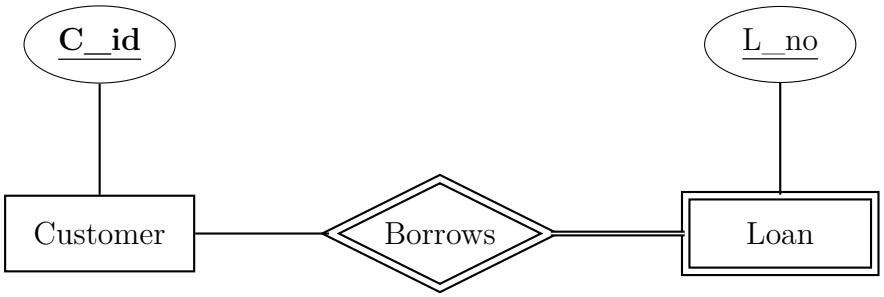

Figure 13: Weak Entity Example: `Loan` depends on `Customer`.

6 Implementation: Solved Example (Complex ERD to Tables)

Instead of memorizing abstract rules, let us walk through a complete, solved example of converting a complex ER Diagram into a Relational Schema (SQL Tables).

6.1 The Scenario: University Academic System

Consider the following ER Diagram representing a university system. It contains:

- **Strong Entities:** `Student` and `Course`.
- **Weak Entity:** `Exam` (Dependent on Course).
- **Multi-valued Attribute:** `Phone` on Student.
- **M:N Relationship:** `Enrolled` (Student takes Course) with a descriptive attribute `Grade`.

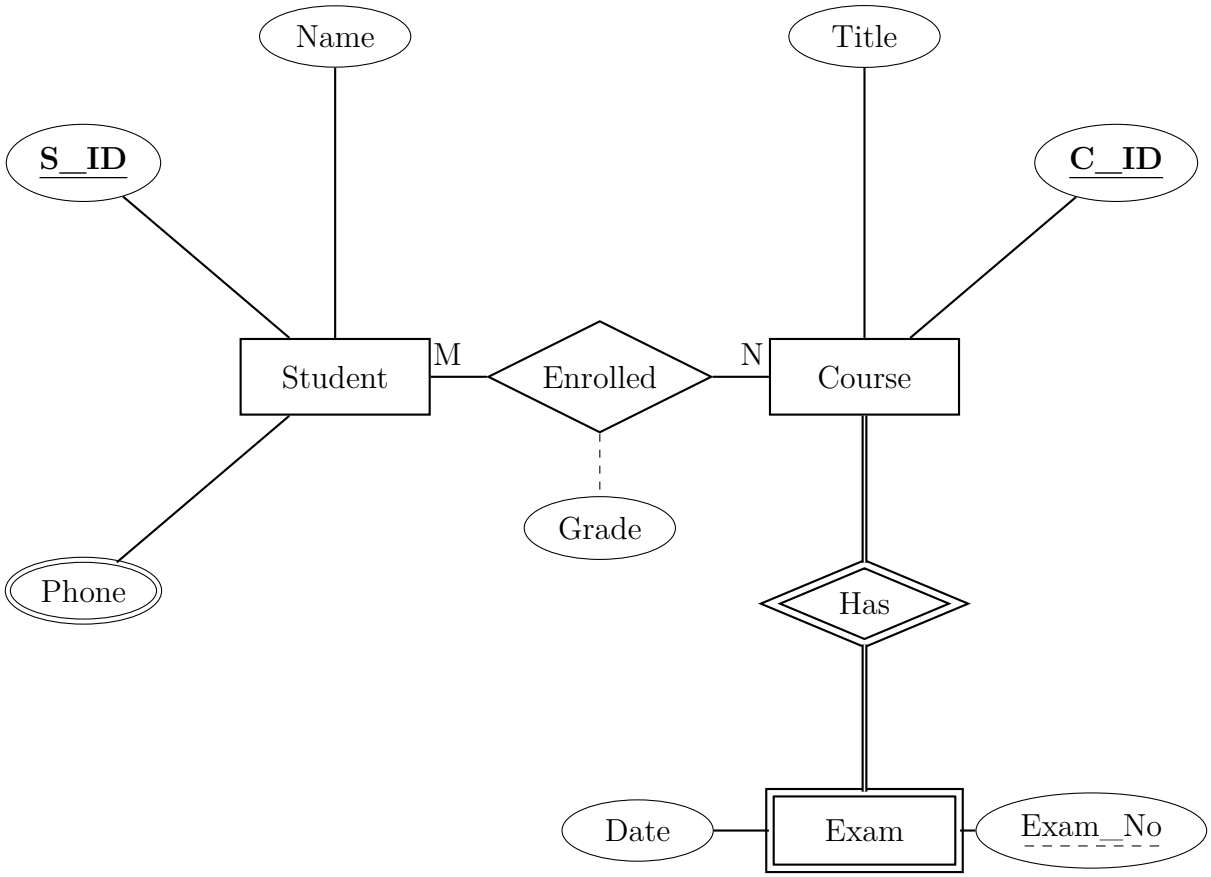

Figure 14: Complex ERD for conversion practice.

6.2 Step-by-Step Conversion

6.2.1 Step 1: Strong Entities

Create tables for independent entities. All simple attributes become columns.

```sql
CREATE TABLE Student (
S_ID INT PRIMARY KEY,
Name VARCHAR(100)
);

CREATE TABLE Course (
C_ID VARCHAR(10) PRIMARY KEY,
Title VARCHAR(100)
);
```

6.2.2 Step 2: Multi-valued Attributes

The `Phone` attribute on `Student` cannot be stored in the main `Student` table (violates 1NF). We create a separate table.

- **Columns:** Owner's Primary Key + The Attribute itself.
- **Composite Primary Key:** (S_ID, Phone).

```sql
CREATE TABLE Student_Phone (
S_ID INT,
Phone_Number VARCHAR(15),
PRIMARY KEY (S_ID, Phone_Number),
FOREIGN KEY (S_ID) REFERENCES Student(S_ID)
);
```

6.2.3 Step 3: Weak Entities

The `Exam` entity relies on `Course`. It has no unique ID of its own, only `Exam_No` (e.g., Exam 1, Exam 2).

- **Foreign Key:** Include `C_ID` from `Course`.
- **Composite Primary Key:** (C_ID, Exam_No).

```sql
CREATE TABLE Exam (
C_ID VARCHAR(10),       -- From Strong Entity
Exam_No INT,            -- Partial Key
Exam_Date DATE,
PRIMARY KEY (C_ID, Exam_No),
FOREIGN KEY (C_ID) REFERENCES Course(C_ID)
ON DELETE CASCADE   -- If Course is deleted, Exams are deleted
);
```

6.2.4 Step 4: Many-to-Many (M:N) Relationship

The `Enrolled` relationship links `Student` and `Course`. Since it is M:N, we cannot simply add a foreign key to either table. We need a **Junction Table**.

- **Columns:** FK from Student + FK from Course + Descriptive Attributes (Grade).
- **Composite Primary Key:** (S_ID, C_ID).

```sql
CREATE TABLE Enrolled (
S_ID INT,
C_ID VARCHAR(10),
Grade CHAR(2),
PRIMARY KEY (S_ID, C_ID),
FOREIGN KEY (S_ID) REFERENCES Student(S_ID),
FOREIGN KEY (C_ID) REFERENCES Course(C_ID)
);
```

Lab VIII

JDBC

1 Lab Workflow

Repeat this subsection for every single lab. If anything in this section fails in IUT lab PCs, check the commands given at the end of the lab manual after notes.

1. Connect to the internet by running `nmtui` in terminal, add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom (including the SQL data file).

3. Disconnect from the **dsl** connection via `nmtui`, connect to the ethernet/wired connection via `nmtui`. If no such connection exists, create an **ethernet** connection and then activate.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. Login to PostgreSQL as 'postgres':

   ```
   sudo -i -u postgres psql
   ```

   2. Create Your Role and Database

   Important: If the role or database already exists, we must drop them first to avoid conflicts.

   ```sql
   -- Terminate active connections to the database
   SELECT pg_terminate_backend(pid)
   FROM pg_stat_activity
   WHERE datname = 'cse4308_12345';

   -- Drop database if it exists
   DROP DATABASE IF EXISTS cse4308_12345;

   -- Drop role if it exists
   DROP ROLE IF EXISTS alice;

   -- Create new role and database
   CREATE ROLE alice LOGIN;
   ALTER ROLE alice WITH PASSWORD 'alice123';

   CREATE DATABASE cse4308_12345 OWNER alice;

   GRANT ALL PRIVILEGES ON DATABASE cse4308_12345 TO alice;
   ```

   This script will:

   - Terminate active sessions to allow safe dropping.
   - Drop the existing `cse4308_12345` database and `alice` role if present.
   - Create a new `alice` role with a password.

- Create the database `cse4308_12345` owned by `alice`.
- Grant all permissions to `alice`.

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

   Now connect to your own database as the ‘alice‘ user.

   1. Connect to Your Database:

```
psql -U alice -d cse4308_12345
```

   Note: You will be prompted for the password. Enter ***`alice123`****.*

   2. Verify the Connection:

```
\conninfo
```

   Expected output:

```
You are connected to database "cse4308_12345" as user "alice".
```

6. **Load the Lab SQL File (Using `\i` command)**

   You must load the table schema and data into your database.

   1. Ensure you are connected to the database (as done in Step 5).
   2. Run the following command inside the `psql` prompt:

```
\i /home/cse/Downloads/lab8_data.sql
```

   Note: The path above assumes you saved the SQL file in the Downloads folder. Adjust the path if you saved it elsewhere.

7. **Verify Tables**

   After loading the SQL file, verify that the tables have been created.

   1. Check Tables:

```
\dt
```

   2. View Table Data:

```
SELECT * FROM account LIMIT 5;
```

2 Database Schema & Data File

Instructions: Copy the code below and save it as a file named `lab8_data.sql` in your `Downloads` directory. This file will be used in Step 6 of the Lab Workflow.

```sql
-- 1. CLEANUP
DROP TABLE IF EXISTS TRANSACTIONS;
DROP TABLE IF EXISTS ACCOUNT;

-- 2. CREATE TABLES
CREATE TABLE ACCOUNT (
A_ID INT PRIMARY KEY
);

CREATE TABLE TRANSACTIONS (
T_ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
DTM TIMESTAMP,
A_ID INT,
AMOUNT DECIMAL(15,2),
TYPE INT, -- 0 for Credit, 1 for Debit
FOREIGN KEY (A_ID) REFERENCES ACCOUNT(A_ID)
);

-- 3. INSERT ACCOUNTS (Generates 100 Accounts)
INSERT INTO ACCOUNT (A_ID)
SELECT generate_series(1, 100);

-- 4. INSERT TRANSACTIONS (Generates ~1000 random transactions)
INSERT INTO TRANSACTIONS (DTM, A_ID, AMOUNT, TYPE)
SELECT
timestamp '2020-01-01' +
random() * (timestamp '2022-12-31' - timestamp '2020-01-01'),
floor(random() * 100 + 1)::int,
(random() * 990000 + 10000)::decimal(15,2),
floor(random() * 2)::int
FROM generate_series(1, 1000);

COMMIT;
```

Listing 6: lab8_data.sql

3 Environment Setup

Since this lab involves Java Database Connectivity (JDBC), additional setup is required beyond the standard PostgreSQL workflow.

3.1 System Installation

Open your terminal (`Ctrl+Alt+T`) and run the following commands to install the JDK and PostgreSQL (if not already installed):

```
# Update package list
sudo apt update

# Install default Java JDK
sudo apt install default-jdk

# Install PostgreSQL
sudo apt install postgresql postgresql-contrib

# Install VS Code (if not installed)
sudo snap install code --classic
```

3.2 VS Code Configuration

1. **Install Java Extensions:**
   - Open VS Code.
   - Click on the Extensions icon (box icon on the left sidebar).
   - Search for **"Extension Pack for Java"** by Microsoft and click Install.
2. **Get the PostgreSQL JDBC Driver:**
   - Download the driver JAR file from: `https://jdbc.postgresql.org/download/`
   - Save `postgresql-42.x.x.jar` to a known location (e.g., `Downloads`).
3. **Create Project & Add Driver:**
   - Press `Ctrl+Shift+P` → `Java: Create Java Project` → `No Build Tools`.
   - In the **Java Projects** view (bottom-left sidebar), find **Referenced Libraries**.
   - Click the + icon and select the `postgresql-42.x.x.jar` file you downloaded.

4 Understanding JDBC

4.1 What is JDBC?

JDBC stands for **Java Database Connectivity**. It is a standard Java API (Application Programming Interface) that allows Java applications to interact with relational databases. Before JDBC, writing database applications was difficult because different databases (like PostgreSQL, MySQL, Oracle) used different proprietary APIs. JDBC provides a uniform interface, meaning you can write one piece of Java code and, by simply changing the "Driver", connect it to any supported database.

4.2 How It Works: The Architecture

The JDBC architecture consists of three main layers working together:

1. **The Java Application (Your Code):** This is the code you write in VS Code. It uses standard Java classes like `Connection`, `Statement`, and `ResultSet`. Importantly, your code does not know the details of how PostgreSQL works internally; it only knows standard Java commands.

2. **The JDBC Driver (The Translator):** This is the `.jar` file you downloaded (e.g., `postgresql-42.x.x.jar`). It acts as a bridge or translator. When your Java code says "Execute Query", the Driver translates that generic request into the specific network protocol that the PostgreSQL server understands.

3. **The Database (PostgreSQL):** The actual storage engine running on your Ubuntu machine. It receives commands from the Driver, executes them, and returns data.

4.3 Core Components & Syntax

To connect to a database, you generally follow these 4 steps, which correspond to specific Java classes:

1. **Load the Driver:** `Class.forName("org.postgresql.Driver");`

2. **Establish Connection:** `DriverManager.getConnection(url, user, pass);`

3. **Create Statement:** `con.createStatement();`

4. **Execute & Process:** `stmt.executeQuery(sql);`

4.4 Prepared Statements

The `Statement` interface is sufficient for simple, static queries (like `SELECT * FROM table`). However, when your application needs to handle User Input (e.g., a user typing their ID to login), you must use a `PreparedStatement`.

Why use PreparedStatement?

1. **Security (SQL Injection Prevention):** If you simply concatenate strings (e.g., `"SELECT * FROM users WHERE name = " + input`), a malicious user can type code instead of a name to hack your database. `PreparedStatement` treats all user input strictly as data, never as executable code.

2. **Performance:** The database pre-compiles the SQL query structure once, making it faster to execute multiple times with different values.

4.5 Real-Life Application

In a real-world scenario, such as an ATM machine:

- When you insert your card (User Input), the Java application reads the card number.
- It uses JDBC to send a secure `SELECT pin FROM users WHERE card_id = ?` query to the bank's database using a PreparedStatement.
- The database returns the result, and the Java app compares it to the PIN you entered.

5 JDBC Code Examples

5.1 Basic Select (Using Statement)

This code demonstrates how to connect to the database and execute a simple, static query.

```java
import java.sql.*;

public class Main {
  public static void main(String[] args) {
    String url = "jdbc:postgresql://localhost:5432/cse4308_12345";
    String username = "alice";
    String password = "alice123";

    try {
      Class.forName("org.postgresql.Driver");
      Connection con = DriverManager.getConnection(url, username, password);
      System.out.println("Connected!");

      Statement statement = con.createStatement();
      ResultSet result = statement.executeQuery("SELECT * FROM account LIMIT 5");

      while (result.next()) {
        System.out.println("Account ID: " + result.getInt("a_id"));
      }

      con.close();
    } catch (Exception e) {
      e.printStackTrace();
    }
  }
}
```

Listing 7: Reading Data (SELECT)

5.2 How to Run the Code

Method 1: VS Code (Recommended)
If you added the JAR file to "Referenced Libraries", click the **Run** button above `main`.

Method 2: Terminal Command (Manual)

```bash
# Compile
javac -cp .:postgresql-42.7.3.jar Main.java
# Run
java -cp .:postgresql-42.7.3.jar Main
```

Lab IX

PL/pgSLQ: Anonymous Blocks

1 Lab Workflow

Repeat this subsection for every single lab. If anything in this section fails in IUT lab PCs, check the commands given at the end of the lab manual after notes.

1. Connect to the internet by running `nmtui` in terminal, go to edit connection and add a dsl connection, then go back and activate that dsl connection. Make sure that your net id is not being used in any other pc or router, if you face any issue, use any other net id from your friends which is not being used.

2. Download all the files posted in the google classroom (including the SQL data file).

3. Disconnect from the **dsl** connection via `nmtui`, connect to the ethernet/wired connection via `nmtui`. If no such connection exists, create an **ethernet** connection and then activate. If any server link is provided, access that server link via any web browser after connecting to the ethernet.

4. **Create Your Role and Database**

   Inside the PostgreSQL shell, create your own role (user) and database.

   1. Login to PostgreSQL as ‘postgres‘:

   ```
   sudo -i -u postgres psql
   ```

   2. Create Your Role and Database

   Important: If the role or database already exists, we must drop them first to avoid conflicts.

   ```sql
   -- Terminate active connections to the database
   SELECT pg_terminate_backend(pid)
   FROM pg_stat_activity
   WHERE datname = 'cse4308_12345';

   -- Drop database if it exists
   DROP DATABASE IF EXISTS cse4308_12345;

   -- Drop role if it exists
   DROP ROLE IF EXISTS alice;

   -- Create new role and database
   CREATE ROLE alice LOGIN;
   ALTER ROLE alice WITH PASSWORD 'alice123';

   CREATE DATABASE cse4308_12345 OWNER alice;

   GRANT ALL PRIVILEGES ON DATABASE cse4308_12345 TO alice;
   ```

   This script will:

   - Terminate active sessions to allow safe dropping.
   - Drop the existing `cse4308_12345` database and `alice` role if present.
   - Create a new `alice` role with password `alice123`.

- Create the database `cse4308_12345` owned by `alice`.
- Grant all permissions to `alice`.

3. Exit PostgreSQL:

```
\q
```

5. **Connect Using Your Newly Created User**

   Now connect to your own database as the ‘alice‘ user.

   1. Connect to Your Database:

```
psql -U alice -d cse4308_12345
```

   Note: You will be prompted for the password. Enter ***`alice123`****.*

   2. Verify the Connection:

```
\conninfo
```

   Expected output:

```
You are connected to database "cse4308_12345" as user "alice".
```

6. **Load the Lab SQL File (Using `\i` command)**

   You must load the table schema and data into your database.

   1. Ensure you are connected to the database (as done in Step 5).
   2. Run the following command inside the `psql` prompt:

```
\i /home/cse/Downloads/setup.sql
```

   Note: The path above assumes you saved the SQL file in the Downloads folder. Adjust the path if you saved it elsewhere.

7. **Verify Tables**

   After loading the SQL file, verify that the tables have been created. Check Tables:

```
\dt
```

==========

2 Database Schema & Data

Instructions: Save this code as `setup.sql` and run it using `\i setup.sql` in your psql terminal. This script generates a dataset of 1,000 employees with varying salaries and departments to simulate a real-world scenario.

```sql
-- 1. CLEANUP
DROP TABLE IF EXISTS employees;

-- 2. CREATE TABLE
CREATE TABLE employees (
employee_id SERIAL PRIMARY KEY,
first_name  VARCHAR(50),
last_name   VARCHAR(50),
salary      NUMERIC(10, 2),
department  VARCHAR(50)
);

-- 3. INSERT KEY PERSONNEL (Deterministic Data)
-- We insert these specific people so we can always find them in queries
INSERT INTO employees (first_name, last_name, salary, department) VALUES
('Alice', 'Director', 25000.00, 'Management'),
('Bob', 'Architect', 15000.00, 'IT'),
('Charlie', 'Manager', 12000.00, 'Marketing'),
('Rahim', 'Uddin', 4500.00, 'Operations');

-- 4. BULK INSERT (Randomized Data)
-- This generates 1000 fake employees
INSERT INTO employees (first_name, last_name, salary, department)
SELECT
-- Pick a random first name from the array (Mixed International & Bangladeshi)
(ARRAY[
'John','Jane','Mike','Sara','Tom','Emily','Chris','Anna','Steve','Laura',
'Rahim','Karim','Fatima','Ayesha','Rafiq','Jabbar','Nasrin','Monir','Sultana','Kamal
  '
])[floor(random() * 20 + 1)],

-- Pick a random last name (Mixed International & Bangladeshi)
(ARRAY[
'Smith','Johnson','Williams','Jones','Brown','Davis','Miller','Wilson','Moore','
  Taylor',
'Ahmed','Hossain','Khan','Chowdhury','Rahman','Islam','Sarker','Uddin','Ali','Hasan'
])[floor(random() * 20 + 1)],

-- Generate random salary between 3,000 and 18,000
ROUND((random() * 15000 + 3000)::numeric, 2),

-- Pick a random department
(ARRAY['IT', 'HR', 'Management', 'Marketing', 'Sales', 'Operations'])[floor(random()
   * 6 + 1)]
FROM generate_series(1, 1000);

-- 5. VERIFY
-- Should return 1004 rows
SELECT count(*) as total_employees FROM employees;
```

Listing 8: setup.sql

3 Introduction to PL/pgSQL

PL/pgSQL (Procedural Language/PostgreSQL Structured Query Language) extends SQL by adding procedural constructs. While SQL is declarative (stating *what* you want), PL/pgSQL allows you to state *how* to achieve it using loops, variables, and logic.

3.1 Anonymous Blocks

An **Anonymous Block** is a piece of code that is executed immediately and not stored in the database. In PostgreSQL, this is done using the `DO` statement.

Structure: The code body must be enclosed in Dollar Quotes (`$$`).

```
DO $$
DECLARE
-- Variable declarations
BEGIN
-- Execution logic
EXCEPTION
-- Error handling
END $$;
```

3.2 Outputting Text

PostgreSQL uses `RAISE NOTICE` to print messages to the console (similar to `System.out.println` in Java or `printf` in C).

4 Variables and Arithmetic

Variables must be declared in the `DECLARE` section. You can initialize them using the `:=` operator.

```
DO $$
DECLARE
v_radius    NUMERIC := 5.5;
v_pi        CONSTANT NUMERIC := 3.14159;
v_area      NUMERIC;
BEGIN
-- Formula: Area = pi * r^2
v_area := v_pi * (v_radius ^ 2);

RAISE NOTICE 'Radius: %', v_radius;
RAISE NOTICE 'Calculated Area: %', v_area;
END $$;
```

Listing 9: Geometry Calculation

5 Single Row Queries (SELECT INTO)

To use data from a table inside a PL/pgSQL block, you cannot simply write `SELECT * FROM table`. The result must be stored in a variable using the `INTO` keyword.

Rule: The query must return exactly **one** row. If it returns zero rows or multiple rows, it may raise an error or warning depending on configuration.

```
DO $$
DECLARE
v_emp_id        INT := 1;
v_full_name     TEXT;
v_salary        NUMERIC;
BEGIN
-- 1. Fetch data into variables
SELECT first_name || ' ' || last_name, salary
INTO v_full_name, v_salary
FROM employees
WHERE employee_id = v_emp_id;

RAISE NOTICE 'Employee: % earns %', v_full_name, v_salary;

-- 2. Use variables in DML (Update)
UPDATE employees
SET salary = salary + 100
WHERE employee_id = v_emp_id;

RAISE NOTICE 'Salary updated.';
END $$;
```

Listing 10: Fetching Employee Data

6 Control Structures

6.1 Conditional Logic (IF/ELSE)

PL/pgSQL supports `IF-THEN-ELSIF-ELSE` logic.

```
DO $$
DECLARE
v_salary NUMERIC;
BEGIN
-- Get Alice's salary
SELECT salary INTO v_salary FROM employees WHERE first_name = 'Alice';

IF v_salary > 10000 THEN
RAISE NOTICE 'Status: Executive Level';
ELSIF v_salary BETWEEN 5000 AND 10000 THEN
RAISE NOTICE 'Status: Manager Level';
ELSE
RAISE NOTICE 'Status: Entry Level';
END IF;
END $$;
```

Listing 11: Salary Evaluation

6.2 CASE Statement

The `CASE` statement is useful for selecting a value based on multiple patterns.

```
DO $$
DECLARE
v_grade CHAR(1) := 'B';
v_result TEXT;
BEGIN
CASE v_grade
WHEN 'A' THEN v_result := 'Excellent';
WHEN 'B' THEN v_result := 'Good';
ELSE v_result := 'Average';
END CASE;

RAISE NOTICE 'Grade B means: %', v_result;
END $$;
```

7 Loops

7.1 WHILE Loop

Executes as long as the condition is true.

```
DO $$
DECLARE
v_counter INT := 1;
BEGIN
WHILE v_counter <= 3 LOOP
RAISE NOTICE 'Count: %', v_counter;
v_counter := v_counter + 1;
```

```
END LOOP;
END $$;
```

7.2 FOR Loop (Iterating Query Results)

This is the most powerful loop in PL/pgSQL. It allows you to iterate over every row returned by a query without explicitly opening or closing cursors.

```
DO $$
DECLARE
r RECORD; -- A generic holder for row data
BEGIN
RAISE NOTICE '--- IT Department Staff ---';

-- "r" becomes a row for every iteration
FOR r IN
SELECT first_name, salary FROM employees WHERE department = 'IT'
LOOP
RAISE NOTICE 'Name: %, Salary: %', r.first_name, r.salary;
END LOOP;
END $$;
```

Listing 12: Looping through rows

8 Using WITH (CTEs) in Blocks

You cannot start a `DO` block with `WITH`. However, you can use Common Table Expressions (CTEs) inside the SQL statements contained within the block.

```
DO $$
DECLARE
r RECORD;
BEGIN
-- We want to find employees who earn more than the average
FOR r IN
WITH AvgSal AS (
SELECT AVG(salary) as val FROM employees
)
SELECT e.first_name, e.salary, a.val as average
FROM employees e, AvgSal a
WHERE e.salary > a.val
LOOP
RAISE NOTICE '% earns % (Above Avg: %)',
r.first_name, r.salary, ROUND(r.average, 2);
END LOOP;
END $$;
```

Listing 13: CTE inside Anonymous Block

Lab X

PL/pgSQL Named Blocks: Functions and Procedures

1 Introduction to Named Blocks

Anonymous blocks (`DO $$`) evaluate procedural logic for single execution but are not persistently stored in the database schema. Persistent named blocks, referred to as subprograms, encapsulate procedural logic for reuse. In PostgreSQL, subprograms are classified into two primary constructs: **User-Defined Functions (UDFs)** and **Stored Procedures**.

2 User-Defined Functions (UDFs)

A User-Defined Function (UDF) is a named PL/pgSQL block that evaluates to a specific return type. Functions are invoked within expressions (e.g., a Data Manipulation Language `SELECT` statement) and structurally require a `RETURN` or `RETURN QUERY` statement. A defining characteristic of a function is that it executes entirely within the transaction scope of the invoking query; it cannot execute Transaction Control Language (TCL) commands such as `COMMIT` or `ROLLBACK`.

2.1 Scalar and Table-Valued Functions

A scalar function evaluates to a single, atomic value per invocation, utilizing the `RETURNS <datatype>` clause. Conversely, a table-valued function evaluates to a relational dataset, utilizing the `RETURNS TABLE` clause and the `RETURN QUERY` statement to populate the output relation.

```
CREATE OR REPLACE FUNCTION calculate_annual_compensation(p_emp_id INT)
RETURNS NUMERIC AS $$
DECLARE
v_monthly_salary NUMERIC;
v_annual_comp NUMERIC;
BEGIN
SELECT salary INTO v_monthly_salary
FROM employees
WHERE employee_id = p_emp_id;

v_annual_comp := (v_monthly_salary * 12) + (v_monthly_salary * 0.5);

RETURN v_annual_comp;
END;
$$ LANGUAGE plpgsql;
```

Listing 14: Scalar Function Definition

3 Stored Procedures

A Stored Procedure is a named PL/pgSQL block invoked via the `CALL` statement. Unlike functions, procedures do not evaluate to a direct return value (omitting the `RETURNS` clause) and are capable of autonomous transaction control. Procedures can initiate and terminate transactions using TCL commands (`COMMIT` and `ROLLBACK`). This capability dictates their use for bulk data manipulation, schema migrations, and batch processing workflows.

```
CREATE OR REPLACE PROCEDURE execute_department_transfer(
p_emp_id INT,
p_new_dept VARCHAR,
p_raise NUMERIC
)
LANGUAGE plpgsql AS $$
BEGIN
UPDATE employees
SET department = p_new_dept, salary = salary + p_raise
WHERE employee_id = p_emp_id;

IF NOT FOUND THEN
RAISE EXCEPTION 'Identifier % does not exist.', p_emp_id;
END IF;

-- Explicit Transaction Control
COMMIT;
END;
$$;
```

Listing 15: Stored Procedure with Transaction Control

4 Parameter Modes and Default Behavior

Formal parameters in PL/pgSQL are assigned a mode that dictates the direction of data binding between the calling environment and the subprogram. The supported modes are `IN`, `OUT`, and `INOUT`.

4.1 Implicit Default Behavior

If a parameter mode is omitted during the formal parameter declaration, the PL/pgSQL compiler implicitly assigns the `IN` mode by default. Consequently, any unannotated parameter is strictly evaluated as a read-only constant within the subprogram's execution scope; attempting to mutate its state will trigger a compilation error.

4.2 Mode Definitions

- **`IN` Mode:** Specifies that the formal parameter accepts an actual argument from the caller. It is lexically immutable within the subprogram body.
- **`OUT` Mode:** Specifies an uninitialized variable bound to the caller's environment. The subprogram assigns a value to it, which is transmitted back upon termination. When `OUT` parameters are present in a function, the `RETURNS` clause is typically omitted as the function implicitly returns a record of the `OUT` variables.
- **`INOUT` Mode:** A hybrid parameter that accepts an initialized actual argument from the caller, permits state mutation within the subprogram body, and transmits the modified state back to the caller upon termination.

5 Integration of Parameter Modes

The following examples demonstrate the simultaneous application of `IN`, `OUT`, and `INOUT` modes within a single subprogram.

5.1 Function Implementation with Mixed Modes

When a function incorporates `OUT` and `INOUT` parameters, it evaluates to a composite `RECORD` type containing the mutated variables.

```
CREATE OR REPLACE FUNCTION evaluate_employee_metrics(
p_emp_id IN INT,                        -- Implicit or explicit IN: Read-only
p_performance_score INOUT NUMERIC,      -- Mutated and returned
p_eligible_for_promotion OUT BOOLEAN    -- Calculated and returned
)
LANGUAGE plpgsql AS $$
DECLARE
v_current_salary NUMERIC;
BEGIN
-- Read the IN parameter
SELECT salary INTO v_current_salary
FROM employees
WHERE employee_id = p_emp_id;
```

```
-- Mutate the INOUT parameter based on internal logic
IF v_current_salary < 10000.00 THEN
p_performance_score := p_performance_score * 1.10;
END IF;

-- Assign the OUT parameter
IF p_performance_score >= 85.00 THEN
p_eligible_for_promotion := TRUE;
ELSE
p_eligible_for_promotion := FALSE;
END IF;

-- No RETURN statement required; OUT/INOUT parameters form the implicit return
  record
END;
$$;
```

Listing 16: Function integrating IN, OUT, and INOUT modes

Invocation:

```
-- The caller provides arguments only for IN and INOUT parameters.
SELECT * FROM evaluate_employee_metrics(1, 80.00);
```

5.2 Procedure Implementation with Mixed Modes

Procedures utilizing mixed parameter modes are invoked via the `CALL` statement. The caller must provide initialized variables for `INOUT` parameters and target variables for `OUT` parameters to capture the modified states.

```
CREATE OR REPLACE PROCEDURE process_annual_review(
p_dept_id IN VARCHAR,              -- Read-only constraint
p_budget_allocation INOUT NUMERIC, -- Mutated and returned to caller
p_review_status OUT VARCHAR        -- Assigned and returned to caller
)
LANGUAGE plpgsql AS $$
DECLARE
v_emp_count INT;
BEGIN
-- Utilize IN parameter for scalar subquery
SELECT COUNT(*) INTO v_emp_count
FROM employees
WHERE department = p_dept_id;

-- Transform the INOUT parameter
p_budget_allocation := p_budget_allocation - (v_emp_count * 1500.00);

-- Initialize the OUT parameter
IF p_budget_allocation > 0 THEN
p_review_status := 'BUDGET_SURPLUS';
ELSE
p_review_status := 'BUDGET_DEFICIT';
END IF;

-- Procedural Transaction Control
COMMIT;
END;
```

```
$$;
```

Listing 17: Procedure integrating IN, OUT, and INOUT modes

Invocation:

```
DO $$
DECLARE
v_dept_budget NUMERIC := 50000.00; -- Initialized for INOUT
v_status VARCHAR;                  -- Uninitialized for OUT
BEGIN
-- Execute the procedure, passing variables by reference
CALL process_annual_review('IT', v_dept_budget, v_status);

RAISE NOTICE 'Remaining Budget: %, Status: %', v_dept_budget, v_status;
END;
$$;
```